Unit-II

- Software Engineering
- Software Process
- The Waterfall Model
- Incremental Process Model
- Evolutionary Process Model
- Spiral Model

Software Engineering:

Software is a program or set of programs containing instructions that provide the desired functionality. Engineering is the process of designing and building something that serves a particular purpose and finds a cost-effective solution to problems.

What is Software Engineering?

Software Engineering is the process of designing, developing, testing, and maintaining software. It is a systematic and disciplined approach to software development that aims to create high-quality, reliable, and maintainable software.

1. Software engineering includes a variety of techniques, tools, and methodologies, including requirements analysis, design, testing, and maintenance.

2. It is a rapidly evolving field, and new tools and technologies are constantly being developed to improve the software development process.

3. By following the principles of software engineering and using the appropriate tools and methodologies, software developers can create high-quality, reliable, and maintainable software that meets the needs of its users.

4. Software Engineering is mainly used for large projects based on software systems rather than single programs or applications.

5. The main goal of Software Engineering is to develop software applications for improving quality, budget, and time efficiency.

6. Software Engineering ensures that the software that has to be built should be consistent, correct, also on budget, on time, and within the required requirements.

Key Principles of Software Engineering

1. **Modularity**: Breaking the software into smaller, reusable components that can be developed and tested independently.

2. **Abstraction**: Hiding the implementation details of a component and exposing only the necessary functionality to other parts of the software.

3. **Encapsulation**: Wrapping up the data and functions of an object into a single unit, and protecting the internal state of an object from external modifications.

4. **Reusability**: Creating components that can be used in multiple projects, which can save time and resources.

5. **Maintenance**: Regularly updating and improving the software to fix bugs, add new features, and address security vulnerabilities.

6. **Testing**: Verifying that the software meets its requirements and is free of bugs.

7. **Design Patterns**: Solving recurring problems in software design by providing templates for solving them.

8. **Agile methodologies:** Using iterative and incremental development processes that focus on customer satisfaction, rapid delivery, and flexibility.

9. **Continuous Integration & Deployment:** Continuously integrating the code changes and deploying them into the production environment.

Main Attributes of Software Engineering

Software Engineering is a systematic, disciplined, quantifiable study and approach to the design, development, operation, and maintenance of a software system. There are four main Attributes of Software Engineering.

1. **Efficiency:** It provides a measure of the resource requirement of a software product efficiently.

2. **Reliability:** It assures that the product will deliver the same results when used in similar working environment.

3. **Reusability:** This attribute makes sure that the module can be used in multiple applications.

4. **Maintainability:** It is the ability of the software to be modified, repaired, or enhanced easily with changing requirements.

Software Process:

Software is the set of instructions in the form of programs to govern the computer system and to process the hardware components. To produce a software product the set of activities is used. This set is called a software process.

What are Software Processes?

Software processes in software engineering refer to the methods and techniques used to develop and maintain software. Some examples of software processes include:

- **Waterfall**: a linear, sequential approach to software development, with distinct phases such as requirements gathering, design, implementation, testing, and maintenance.

- **Agile**: a flexible, iterative approach to software development, with an emphasis on rapid prototyping and continuous delivery.

- **Scrum**: a popular Agile methodology that emphasizes teamwork, iterative development, and a flexible, adaptive approach to planning and management.

- **DevOps**: a set of practices that aims to improve collaboration and communication between development and operations teams, with an emphasis on automating the software delivery process.

Each process has its own set of advantages and disadvantages, and the choice of which one to use depends on the specific project and organization.

Components of Software

There are three main components of the software:

1. **Program:** A computer program is a list of instructions that tell a computer what to do.

2. **Documentation:** Source information about the product contained in design documents, detailed code comments, etc.

3. **Operating Procedures:** Set of step-by-step instructions compiled by an organization to help workers carry out complex routine operations.

Other Software Components

Other Software Components are:

1. **Code**: the instructions that a computer executes in order to perform a specific task or set of tasks.

2. **Data**: the information that the software uses or manipulates.

3. **User** interface: the means by which the user interacts with the software, such as buttons, menus, and text fields.

4. **Libraries**: pre-written code that can be reused by the software to perform common tasks.

5. **Documentation**: information that explains how to use and maintain the software, such as user manuals and technical guides.

6. **Test cases**: a set of inputs, execution conditions, and expected outputs that are used to test the software for correctness and reliability.

7. **Configuration files**: files that contain settings and parameters that are used to configure the software to run in a specific environment.

8. **Build and deployment** scripts: scripts or tools that are used to build, package, and deploy the software to different environments.

9. **Metadata**: information about the software, such as version numbers, authors, and copyright information.

Waterfall Model –

The Waterfall Model is a classical software development methodology. It was first introduced by Winston W. Royce in 1970. It is a linear and sequential approach to software development that consists of several phases. It must be completed in a specific order. This classical waterfall model is simple and idealistic. It was once very popular. Today, it is not that popularly used. However, it is important because most other types of software development life cycle models are a derivative of this. In this article we will see waterfall model in detail.

What is the SDLC Waterfall Model?

The waterfall model is a software development model used in the context of large, complex projects, typically in the field of information technology. It is characterized by a structured, sequential approach to project management and software development.

The waterfall model is useful in situations where the project requirements are well-defined and the project goals are clear. It is often used for large-scale projects with long timelines, where there is little room for error and the project stakeholders need to have a high level of confidence in the outcome.

Features of Waterfall Model

Following are the features of the waterfall model:

1. **Sequential Approach**: The waterfall model involves a sequential approach to software development, where each phase of the project is completed before moving on to the next one.

2. **Document-Driven:** The waterfall model depended on documentation to ensure that the project is well-defined and the project team is working towards a clear set of goals.

3. **Quality Control:** The waterfall model places a high emphasis on quality control and testing at each phase of the project, to ensure that the final product meets the requirements and expectations of the stakeholders.

4. **Rigorous Planning**: The waterfall model involves a careful planning process, where the project scope, timelines, and deliverables are carefully defined and monitored throughout the project lifecycle.

Overall, the waterfall model is used in situations where there is a need for a highly structured and systematic approach to software development. It can be effective in ensuring that large, complex projects are completed on time and within budget, with a high level of quality and customer satisfaction.

Importance of Waterfall Model

Following are the importance of waterfall model:

1. **Clarity and Simplicity:** The linear form of the Waterfall Model offers a simple and unambiguous foundation for project development.

2. **Clearly Defined Phases:** The Waterfall Model phases each have unique inputs and outputs, guaranteeing a planned development with obvious checkpoints.

3. **Documentation:** A focus on thorough documentation helps with software comprehension, maintenance, and future growth.

4. **Stability in Requirements:** Suitable for projects when the requirements are clear and stable, reducing modifications as the project progresses.

5. **Resource Optimization:** It encourages effective task-focused work without continuously changing contexts by allocating resources according to project phases.

6. **Relevance for Small Projects:** Economical for modest projects with simple specifications and minimal complexity.

Phases of Waterfall Model

The Waterfall Model has six phases which are:

1. **Requirements:** The first phase involves gathering requirements from stakeholders and analyzing them to understand the scope and objectives of the project.

2. **Design:** Once the requirements are understood, the design phase begins. This involves creating a detailed design document that outlines the software architecture, user interface, and system components.

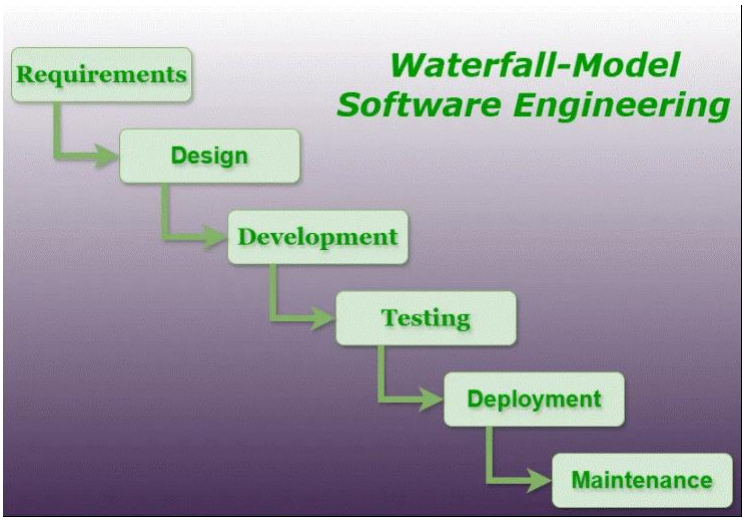
3. **Development:** The Development phase include implementation involves coding the software based on the design specifications. This phase also includes unit testing to ensure that each component of the software is working as expected.

4. **Testing:** In the testing phase, the software is tested as a whole to ensure that it meets the requirements and is free from defects.

5. **Deployment:** Once the software has been tested and approved, it is deployed to the production environment.

6. **Maintenance:** The final phase of the Waterfall Model is maintenance, which involves fixing any issues that arise after the software has been deployed and ensuring that it continues to meet the requirements over time.

The classical waterfall model divides the life cycle into a set of phases. This model considers that one phase can be started after the completion of the previous phase. That is the output of one phase will be the input to the next phase. Thus the development process can be considered as a sequential flow in the waterfall. Here the phases do not overlap with each other. The different sequential phases of the classical waterfall model are shown in the below figure.



Waterfall Model-Software Engineering

Let us now learn about each of these phases in detail which include further phases.

1. Feasibility Study

The main goal of this phase is to determine whether it would be financially and technically feasible to develop the software. The feasibility study involves understanding the problem and then determining the various possible strategies to solve the problem. These different identified solutions are analyzed based on their benefits and drawbacks. The best solution is chosen and all the other phases are carried out as per this solution strategy.

2. Requirements Analysis and Specification

The requirement analysis and specification phase aims to understand the exact requirements of the customer and document them properly. This phase consists of two different activities.

- **Requirement gathering and analysis:** Firstly all the requirements regarding the software are gathered from the customer and then the gathered requirements are analyzed. The goal of the analysis part is to remove incompleteness (an incomplete requirement is one in which some parts of the actual requirements have been omitted) and inconsistencies (an inconsistent requirement is one in which some part of the requirement contradicts some other part).

- **Requirement specification:** These analyzed requirements are documented in a software requirement specification (SRS) document. SRS document serves as a contract between the development team and customers. Any future dispute between the customers and the developers can be settled by examining the SRS document.

3. Design

The goal of this phase is to convert the requirements acquired in the SRS into a format that can be coded in a programming language. It includes high-level and detailed design as well as the overall software architecture. A Software Design Document is used to document all of this effort (SDD).

4. Coding and Unit Testing

In the coding phase software design is translated into source code using any suitable programming language. Thus each designed module is coded. The unit testing phase aims to check whether each module is working properly or not.

5. Integration and System testing

Integration of different modules is undertaken soon after they have been coded and unit tested. Integration of various modules is carried out incrementally over several steps. During each integration step, previously planned modules are added to the partially integrated system and the resultant system is tested. Finally, after all the modules have been successfully integrated and tested, the full working system is obtained and system testing is carried out on this. System testing consists of three different kinds of testing activities as described below.

- **Alpha testing:** Alpha testing is the system testing performed by the development team.

- **Beta testing:** Beta testing is the system testing performed by a friendly set of customers.

- **Acceptance testing:** After the software has been delivered, the customer performs acceptance testing to determine whether to accept the delivered software or reject it.

6. Maintenance

Maintenance is the most important phase of a software life cycle. The effort spent on maintenance is 60% of the total effort spent to develop a full software. There are three types of maintenance.

- **Corrective Maintenance:** This type of maintenance is carried out to correct errors that were not discovered during the product development phase.

- **Perfective Maintenance:** This type of maintenance is carried out to enhance the functionalities of the system based on the customer's request.

- **Adaptive Maintenance:** Adaptive maintenance is usually required for porting the software to work in a new environment such as working on a new computer platform or with a new operating system.

Advantages of Waterfall model

- The waterfall model is simple and easy to understand, to implement, and use.
- All the requirements are known at the beginning of the project, hence it is easy to manage.
- It avoids overlapping of phases because each phase is completed at once.
- This model works for small projects where the requirements are easily understood.
- This model is preferred for those projects where the quality is more important as compared to the cost of the project.

Disadvantages of the Waterfall model

- This model is not good for complex and object oriented projects.
- In this model, the changes are not permitted so it is not fit for moderate to high risk changes in project.
- It is a poor model for long duration projects.
- The problems with this model are uncovered, until the software testing.
- The amount of risk is high.

Incremental Process Model:

The incremental process model is also known as the Successive version model.

First, a simple working system implementing only a few basic features is built and then that is delivered to the customer. Then thereafter many successive iterations/ versions are implemented and delivered to the customer until the desired system is released.

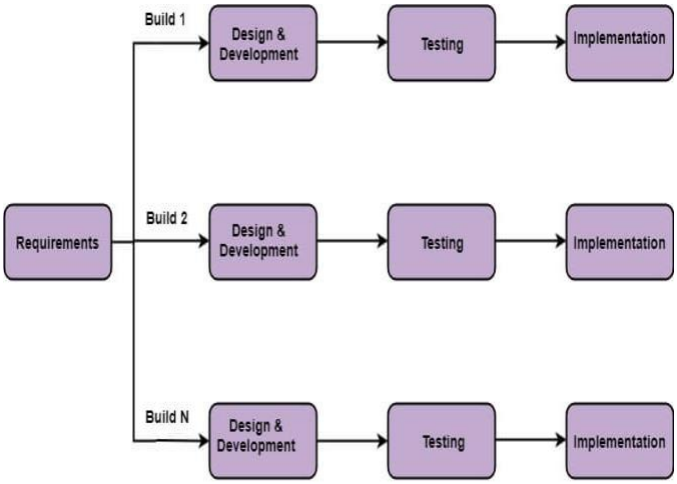


Fig: Incremental Model

d.

1. Requirement analysis: In the first phase of the incremental model, the product analysis expertise identifies the requirements. And the system functional requirements are understood by the requirement analysis team. To develop the software under the incremental model, this phase performs a crucial role.

2. Design & Development: In this phase of the Incremental model of SDLC, the design of the system functionality and the development method are finished with success. When software develops new practicality, the incremental model uses style and development phase.

3. Testing: In the incremental model, the testing phase checks the performance of each existing function as well as additional functionality. In the testing phase, the various methods are used to test the behavior of each task.

4. Implementation: Implementation phase enables the coding phase of the development system. It involves the final coding that design in the designing and development phase and tests the functionality in the testing phase. After completion of this phase, the number of the product working is enhanced and upgraded up to the final system product.

Advantages:

- Errors are easy to be recognized.
- Easier to test and debug
- More flexible.
- Simple to manage risk because it handled during its iteration.
- The Client gets important functionality early.

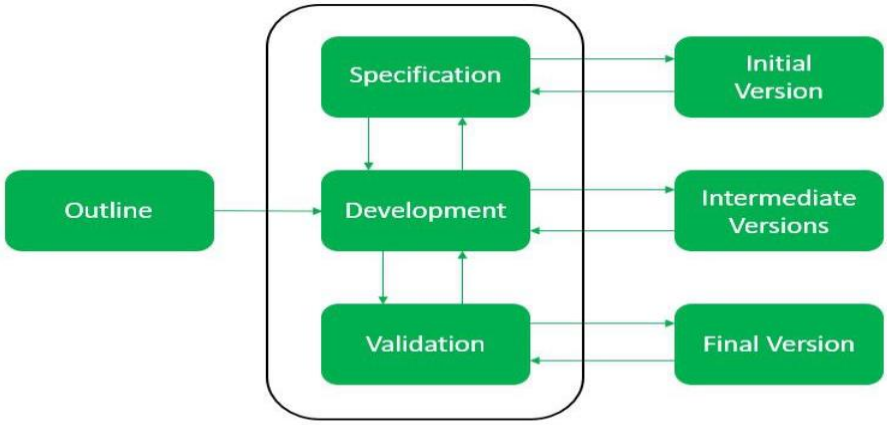
Disadvantages:

- Need for good planning
- Total Cost is high.

- Well defined module interfaces are needed.

Evolutionary Process Model

The evolutionary model is based on the concept of making an initial product and then evolving the software product over time with iterative and incremental approaches with proper feedback. In this type of model, the product will go through several iterations and come up when the final product is built through multiple iterations. The development is carried out simultaneously with the feedback during the development. This model has a number of advantages such as customer involvement, taking feedback from the customer during development, and building the exact product that the user wants. Because of the multiple iterations, the chances of errors get reduced and the reliability and efficiency will increase.



The evolutionary model is normally useful for very large products, where it is easier to find modules for incremental implementation.

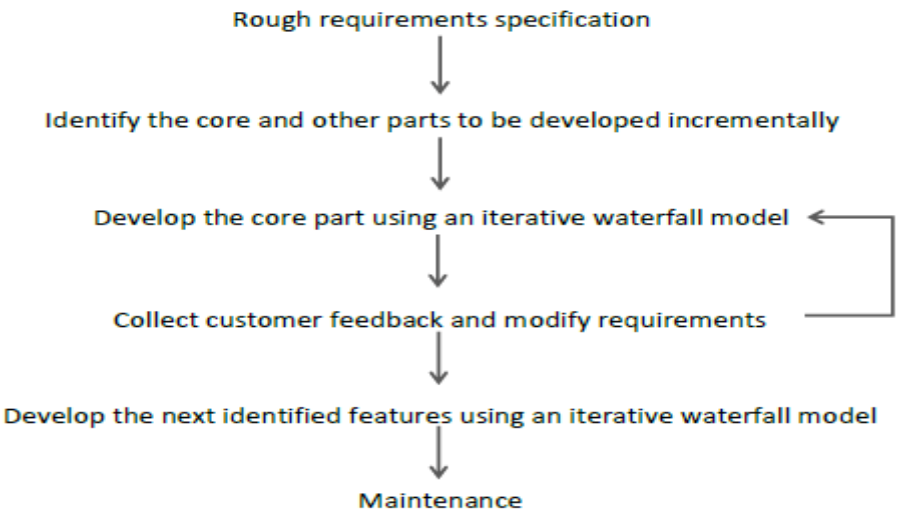


Figure 6: Evolutionary Model of Software Development

Evolutionary Model of Software Development

Often, evolutionary model is used when the customer prefers to receive the product in increments so that he can start using the different features as and when they are developed rather than waiting all the time for the full product to be developed and delivered.

Advantages :

- Allows for incremental development and improvement
- Enables early delivery of working software
- Facilitates continuous feedback and collaboration with stakeholders
- Reduces the risk of project failure by identifying issues early on

Disadvantages :

- May require more time and effort to manage and coordinate changes
- Requires a high level of collaboration and communication among team members
- May not be suitable for projects with strict deadlines or fixed budgets
- Can result in a less predictable and more uncertain development process

Spiral Model

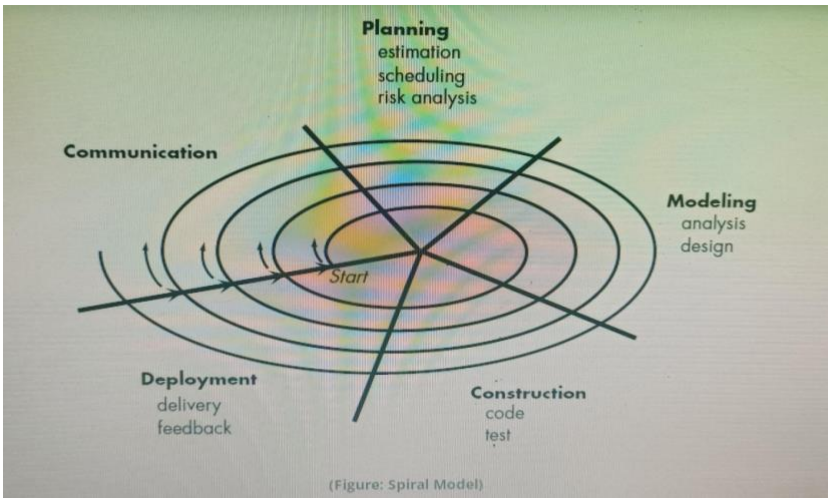
The spiral model is a software development process model that combines elements of both waterfall and iterative models. It is characterized by a series of iterations, or spirals, where each spiral represents a phase of the software development life cycle (SDLC). The model is based on the idea of risk management, where the project is divided into smaller iterations and each iteration includes planning, risk analysis, development, and evaluation. The spiral model allows for flexibility and adaptability, as it allows for

changes to be made at each iteration based on the feedback and evaluation of the previous iteration. This iterative approach helps in managing risks effectively and ensures that the final product meets the desired requirements.

Spiral Model = Concept of Waterfall lifecycle+ Iterative model.
Here are the major characteristics of a spiral model:
This model is risk-driven

- In case of any risk analysis, an alternate solution can be taken into consideration by suggesting and implementing it

- This model is being used to implement large projects.



(Figure: Spiral Model)

Advantages :

- The development team develops the software early.
- This model is very efficient for tedious projects.
- It allows documentation control.
- After each iteration alignment with user needs is ensured for the business goals.

Disadvantages:

- It can be complex and difficult to manage.
- It can be time-consuming and expensive.
- It is not suitable for small and simple projects.
- It can be difficult to estimate the cost and schedule of a spiral project.

****

Best of Luck!